

I-Logical Approaches Circuits

Foundations of Neuro-Symbolic AI

Alex Goessmann

University of Applied Science Würzburg-Schweinfurt (THWS)

June 1, 2026

Outline

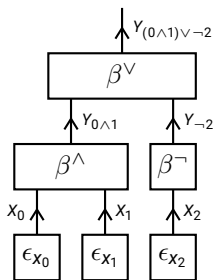
- ▶ Circuit computation model
- ▶ Tensor representation
- ▶ Example: m -adic sums

Recall: Basis calculus in propositional logic

Evaluation of propositional formulas at states in basis calculus:

- ▶ Start with the one-hot encoding state to be evaluated
- ▶ Evaluate iteratively sub-formulas, pass the result to the more complex sub-formulas

Example: The formula $(X_0 \wedge X_1) \vee \neg X_2$ is evaluated at a state $(x_0, x_1, x_2) \in [2]^3$ by

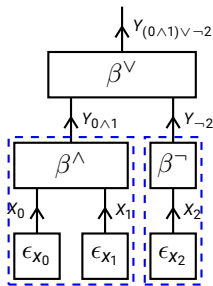


Recall: Basis calculus in propositional logic

Evaluation of propositional formulas at states in basis calculus:

- ▶ Start with the one-hot encoding state to be evaluated
- ▶ Evaluate iteratively sub-formulas, pass the result to the more complex sub-formulas

Example: The formula $(X_0 \wedge X_1) \vee \neg X_2$ is evaluated at a state $(x_0, x_1, x_2) \in [2]^3$ by

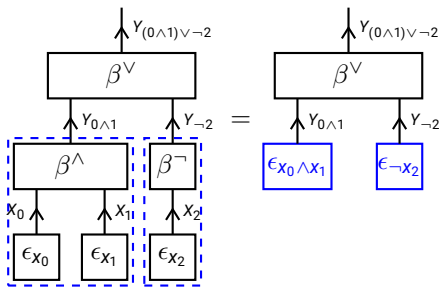


Recall: Basis calculus in propositional logic

Evaluation of propositional formulas at states in basis calculus:

- ▶ Start with the one-hot encoding state to be evaluated
- ▶ Evaluate iteratively sub-formulas, pass the result to the more complex sub-formulas

Example: The formula $(X_0 \wedge X_1) \vee \neg X_2$ is evaluated at a state $(x_0, x_1, x_2) \in [2]^3$ by

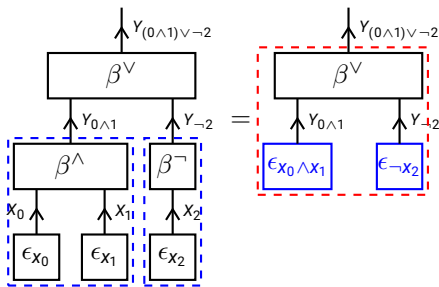


Recall: Basis calculus in propositional logic

Evaluation of propositional formulas at states in basis calculus:

- ▶ Start with the one-hot encoding state to be evaluated
- ▶ Evaluate iteratively sub-formulas, pass the result to the more complex sub-formulas

Example: The formula $(X_0 \wedge X_1) \vee \neg X_2$ is evaluated at a state $(x_0, x_1, x_2) \in [2]^3$ by

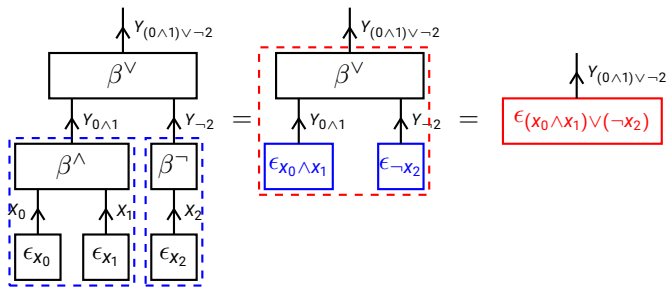


Recall: Basis calculus in propositional logic

Evaluation of propositional formulas at states in basis calculus:

- ▶ Start with the one-hot encoding state to be evaluated
- ▶ Evaluate iteratively sub-formulas, pass the result to the more complex sub-formulas

Example: The formula $(X_0 \wedge X_1) \vee \neg X_2$ is evaluated at a state $(x_0, x_1, x_2) \in [2]^3$ by

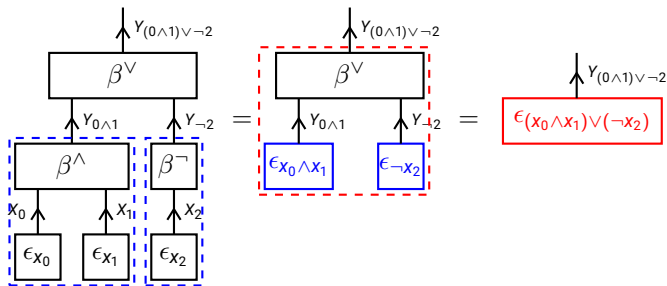


Recall: Basis calculus in propositional logic

Evaluation of propositional formulas at states in basis calculus:

- ▶ Start with the one-hot encoding state to be evaluated
- ▶ Evaluate iteratively sub-formulas, pass the result to the more complex sub-formulas

Example: The formula $(X_0 \wedge X_1) \vee \neg X_2$ is evaluated at a state $(x_0, x_1, x_2) \in [2]^3$ by



Can we generalize to arbitrary functions and find tensor network decompositions based on circuits?

Generalization to circuits

A generic tensor has arbitrary leg dimensions and represents a function

$$q : x_{[d]} \rightarrow \tau [X_{[d]} = x_{[d]}]$$

How to find a decomposition of a tensor into smaller tensors, based on a syntactical description of the function represented by the tensor?

Generalizations required:

- ▶ Arbitrary state numbers to variables: Leaving the interpretation of the states $[2]$ as $\{\text{False}, \text{True}\}$, use instead state sets $[m]$
- ▶ Arbitrary functions applied on states

Circuits

Definition (Circuit)

A circuit is a decorated hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where:

- ▶ Each node $v \in \mathcal{V}$ is decorated by a dimension $m_v \in \mathbb{N}$
- ▶ Each edge $e \in \mathcal{E}$ is a tuple of incoming nodes e^{in} and outgoing nodes e^{out}
- ▶ Each edge $e \in \mathcal{E}$ is decorated by a function

$$q_e : \prod_{v \in e^{\text{in}}} [m_v] \rightarrow \prod_{v \in e^{\text{out}}} [m_v]$$

We assume that:

- ▶ The hypergraph is acyclic
- ▶ Each node appears at most once as an outgoing node

Functions represented by circuits

We denote by

- ▶ $\mathcal{V}^{\text{in}}$ the nodes appearing never as outgoing nodes.
- ▶ $\mathcal{V}^{\text{out}}$ the nodes appearing never as incoming nodes.

Now we can iteratively define a function

$$g^v : \prod_{\tilde{v} \in \mathbf{e}^{\text{in}}} [m_{\tilde{v}}] \rightarrow [m_v]$$

as the composition of the functions decorating the edges, mapping the states of $\mathcal{V}^{\text{in}}$ to the states of $\mathcal{V}^{\text{out}}$, by

$$g^v = q_{e|v} \circ \left(\prod_{\tilde{v} \in \mathbf{e}^{\text{in}}} g^{\tilde{v}} \right).$$

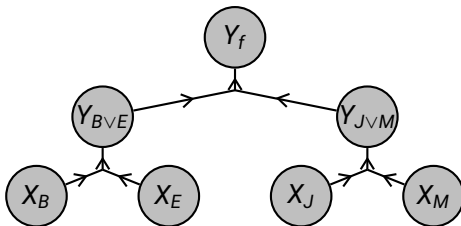
The circuit represents the function

$$g^{\mathcal{G}} = \prod_{v \in \mathcal{V}^{\text{out}}} g^v.$$

Example: Propositional syntax

Consider the formula $(X_B \vee X_E) \Rightarrow (X_J \vee X_M)$

- ▶ The nodes are the variables X_B, X_E, X_J, X_M , the sub-formulas $Y_{B \vee E}, Y_{J \vee M}$, and the formula Y_f
- ▶ The nodes $\mathcal{V}^{\text{in}} = \{X_B, X_E, X_J, X_M\}$ do not appear as outgoing nodes
- ▶ The node Y_f is the only one not appearing as incoming nodes, thus $\mathcal{V}^{\text{out}} = \{Y_f\}$



Outline

- ▶ Circuit computation model
- ▶ Tensor representation
- ▶ Example: m -adic sums

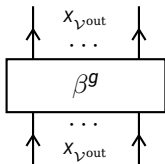
General basis encodings

Consider an arbitrary function

$$g : \bigtimes_{v \in \mathcal{V}^{\text{in}}} [m_v] \rightarrow \bigtimes_{v \in \mathcal{V}^{\text{out}}} [m_v] .$$

We define its **basis encoding** as the tensor

$$\beta^g [X_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}}] = \sum_{x_{\mathcal{V}^{\text{in}}} \in \times_{v \in \mathcal{V}^{\text{in}}} [m_v]} \epsilon_{g(x_{\mathcal{V}^{\text{in}})} [X_{\mathcal{V}^{\text{out}}}] \otimes \epsilon_{x_{\mathcal{V}^{\text{in}}} [X_{\mathcal{V}^{\text{in}}}] .$$



Decomposition into tensor networks

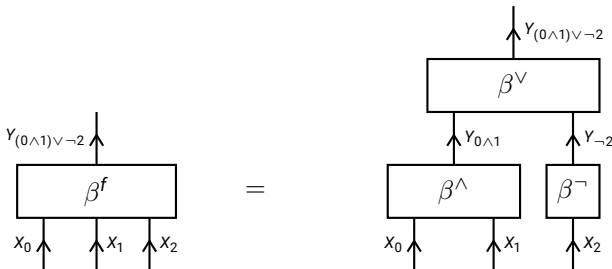
To each edge e we choose a basis encoding

$$\beta^{q_e} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] = \sum_{x_{e^{\text{in}}} \in X_{v \in e^{\text{in}}} [m_v]} \epsilon_{q_e(x_{e^{\text{in}}})} [X_{e^{\text{out}}}] \otimes \epsilon_{x_{e^{\text{in}}}} [X_{e^{\text{in}}}]$$

The basis encoding to a function of a circuit is decomposed into basis encodings of the node functions:

$$\beta^{q_g} [X_{v^{\text{out}}}, X_{v^{\text{in}}}] = \langle \{ \beta^{q_e} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] : e \in \mathcal{E} \} \rangle [X_{v^{\text{out}}}, X_{v^{\text{in}}}]$$

Example: The basis encoding of the formula $f = (X_0 \wedge X_1) \vee \neg X_2$ is decomposed into the basis encodings of the connectives $\wedge, \vee, \neg$ as



Outline

- ▶ Circuit computation model
- ▶ Tensor representation
- ▶ Example: m -adic sums

***m*-adic representation of integers**

Let m, d be arbitrary natural numbers. Any number $n \in [m^d]$ has a unique m -adic representation

$$\sum_{k \in [d]} x_k \cdot m^k$$

where for each $k \in [d]$

$$x_k \in [m].$$

Example: For $m = 3$ and $d = 2$, the number $5 \in [9]$ has the 3-adic representation

$$(x_0, x_1) = (2, 1)$$

since

$$5 = 2 \cdot 3^0 + 1 \cdot 3^1.$$

Summation function

We want to find a circuit decomposing the summation function

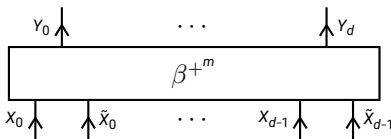
$$+^m : [m^d] \times [m^d] \rightarrow [m^{d+1}]$$

defined for tuples $(x_{[d]}, \tilde{x}_{[d]}) \in [m]^d$ with $+^m(x_{[d]}, \tilde{x}_{[d]}) = y_{[d+1]}$ as

$$\sum_{k \in [d+1]} y_k \cdot m^k = \sum_{k \in [d]} (x_k + \tilde{x}_k) \cdot m^k.$$

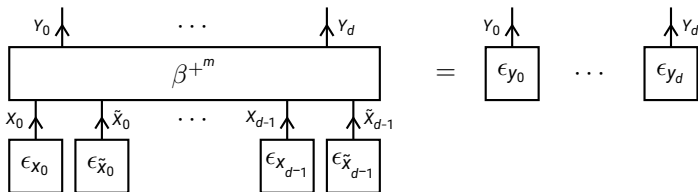
The basis encoding of the sum is the tensor

$$\begin{aligned} & \beta^{+^m} [Y_{[d+1]}, X_{[d]}, \tilde{X}_{[d]}] \\ &= \sum_{x_{[d]}, \tilde{x}_{[d]}} \epsilon_{+^m(x_{[d]}, \tilde{x}_{[d]})} [Y_{[d+1]}] \otimes \epsilon_{x_{[d]}} [X_{[d]}] \otimes \epsilon_{\tilde{x}_{[d]}} [\tilde{X}_{[d]}]. \end{aligned}$$



Integer summation in basis calculus

Basis calculus: Contraction of the basis encoding with the one-hot encoded input computes the one-hot encoding of the summation



We therefore compute by contraction the unique $y_{[d+1]}$ such that

$$\sum_{k \in [d]} (x_k + \tilde{x}_k) \cdot m^k = \sum_{k \in [d+1]} y_k \cdot m^k$$

How to perform this contraction more efficiently?

Bitwise computation of m -adic sums

Example: Summation of 14 and 15 using the basis $m = 3$.

We have a representation by $x_{[3]} = (2, 1, 1)$ and $\tilde{x}_{[3]} = (0, 2, 1)$ since

$$14 = 1 \cdot 3^2 + 1 \cdot 3^1 + 2 \cdot 3^0$$

$$15 = 1 \cdot 3^2 + 2 \cdot 3^1 + 0 \cdot 3^0$$

The summation can be performed locally using carry bits:

$$\begin{array}{rcccc} & 1 & 1 & 2 & \\ & 1 & 2 & 0 & \\ \hline * & * & * & * & \end{array}$$

Bitwise computation of m -adic sums

Example: Summation of 14 and 15 using the basis $m = 3$.

We have a representation by $x_{[3]} = (2, 1, 1)$ and $\tilde{x}_{[3]} = (0, 2, 1)$ since

$$14 = 1 \cdot 3^2 + 1 \cdot 3^1 + 2 \cdot 3^0$$

$$15 = 1 \cdot 3^2 + 2 \cdot 3^1 + 0 \cdot 3^0$$

The summation can be performed locally using carry bits:

$$\begin{array}{rcccc} & 1 & 1 & 2 & \\ & 1 & 2_0 & 0 & \\ \hline * & * & * & 2 & \end{array}$$

Bitwise computation of m -adic sums

Example: Summation of 14 and 15 using the basis $m = 3$.

We have a representation by $x_{[3]} = (2, 1, 1)$ and $\tilde{x}_{[3]} = (0, 2, 1)$ since

$$14 = 1 \cdot 3^2 + 1 \cdot 3^1 + 2 \cdot 3^0$$

$$15 = 1 \cdot 3^2 + 2 \cdot 3^1 + 0 \cdot 3^0$$

The summation can be performed locally using carry bits:

$$\begin{array}{r} 1 \quad 1 \quad 2 \\ 1_1 \quad 2_0 \quad 0 \\ \hline * \quad * \quad 0 \quad 2 \end{array}$$

Bitwise computation of m -adic sums

Example: Summation of 14 and 15 using the basis $m = 3$.

We have a representation by $x_{[3]} = (2, 1, 1)$ and $\tilde{x}_{[3]} = (0, 2, 1)$ since

$$14 = 1 \cdot 3^2 + 1 \cdot 3^1 + 2 \cdot 3^0$$

$$15 = 1 \cdot 3^2 + 2 \cdot 3^1 + 0 \cdot 3^0$$

The summation can be performed locally using carry bits:

$$\begin{array}{r} \\ \\ \hline * \end{array}$$

Bitwise computation of m -adic sums

Example: Summation of 14 and 15 using the basis $m = 3$.

We have a representation by $x_{[3]} = (2, 1, 1)$ and $\tilde{x}_{[3]} = (0, 2, 1)$ since

$$14 = 1 \cdot 3^2 + 1 \cdot 3^1 + 2 \cdot 3^0$$

$$15 = 1 \cdot 3^2 + 2 \cdot 3^1 + 0 \cdot 3^0$$

The summation can be performed locally using carry bits:

$$\begin{array}{r} \\ \\ \hline 1 \end{array}$$

Bitwise computation of m -adic sums

Example: Summation of 14 and 15 using the basis $m = 3$.

We have a representation by $x_{[3]} = (2, 1, 1)$ and $\tilde{x}_{[3]} = (0, 2, 1)$ since

$$14 = 1 \cdot 3^2 + 1 \cdot 3^1 + 2 \cdot 3^0$$

$$15 = 1 \cdot 3^2 + 2 \cdot 3^1 + 0 \cdot 3^0$$

The summation can be performed locally using carry bits:

$$\begin{array}{r} 1 2 \\ 1_1 2_0 0 \\ \hline 1 0 2 \end{array}$$

We computed the result

$$1 \cdot 3^3 + 0 \cdot 3^2 + 0 \cdot 3^1 + 2 \cdot 3^0 = 29.$$

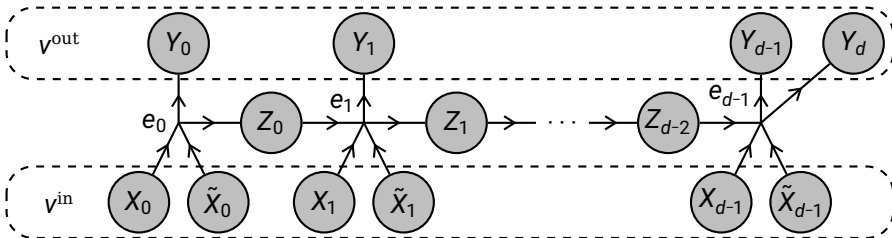
Circuit of half-adders

Nodes representing the bits to numbers:

- ▶ $X_{[d]}$ representing the first number to be summed
- ▶ $\tilde{X}_{[d]}$ representing the second number to be summed
- ▶ $Y_{[d+1]}$ representing the sum
- ▶ $Z_{[d-1]}$ representing carry bits

Edges are enumerated by $[d-1] = \{0, \dots, d-2\}$ representing half-adders:

$$q_{e_k}(z_{k-1}, x_k, \tilde{x}_k) = \left((z_{k-1} + x_k + \tilde{x}_k) \bmod m, \left\lfloor \frac{z_{k-1} + x_k + \tilde{x}_k}{m} \right\rfloor \right)$$

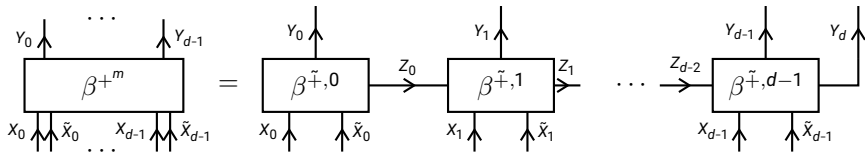


Decomposition of the basis encoding

We have a decomposition of the basis encoding of the sum as

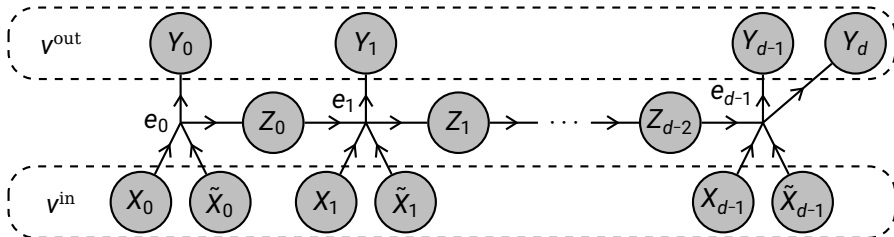
$$\begin{aligned} & \beta^{+m} \left[Y_{[d+1]}, X_{[d]}, \tilde{X}_{[d]} \right] \\ &= \langle \{ \beta^{\tilde{+},0} \left[Y_0, Z_0, X_0, \tilde{X}_0 \right] \} \cup \\ & \quad \{ \beta^{\tilde{+},k} \left[Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1} \right] : k \in \{1, \dots, d-2\} \} \cup \\ & \quad \{ \beta^{\tilde{+},d-2} \left[Y_{d-1}, Y_d, X_{d-1}, \tilde{X}_{d-1}, Z_{d-2} \right] \} \rangle \left[Y_{[d+1]}, X_{[d]}, \tilde{X}_{[d]} \right]. \end{aligned}$$

As a tensor network diagram this is visualized by:



Summary: Decomposition of the basis encoding based on a circuit

a) Hypergraph representing the circuit:



b) Basis encoding with decomposition:

